

Class 12: Advanced React / Next.js

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

Redux



- Redux เป็น pattern และ library สำหรับ manage และ update application state โดยใช้ event ต่าง ๆ ที่เรียกว่า action
  - Redux ทำหน้าที่เป็น store กลางสำหรับจัดเก็บ state ที่ share ใช้งานทั่วทั้ง application โดยจัดการ Global State
  - Redux มาพร้อม rule ต่าง ๆ เพื่อจัดการให้การ update state กระทำในลักษณะที่คาดการณ์ได้ (predictable)
    - Redux มาพร้อม pattern และ tool ต่าง ๆ ที่ช่วยให้ developer เข้าใจการ update ของ state ใน application ว่าเกิดขึ้นเมื่อไร (when) ที่ไหน (where) เพราะเหตุใด (why) และอย่างไร (how) และเข้าใจพฤติกรรมของ application logic เมื่อเกิดการ update เหล่านั้น
    - เอื้อต่อการ coding ที่คาดการณ์ได้ (predictable) และทดสอบได้ ให้พฤติกรรมที่เป็นไปตามความคาดหวัง
  - Redux Toolkit เป็น Tools ที่ให้รูปแบบใหม่ในการพัฒนา Redux logic
    - มาพร้อม package และ function จำเป็นสำหรับการสร้าง Redux app
    - การใช้ Redux Toolkit เป็น best practice สำหรับการใช้งาน Redux ทำให้ทำงานกับ Redux ง่ายขึ้น และหลีกเลี่ยงข้อผิดพลาด

Redux Persist



- Redux Persist เป็น library ที่รองรับการบันทึก Redux store ลงใน persistent storage เช่น ใน local storage ของ application
  - ข้อมูลใน Redux store ยังคงถูกรักษาไว้ (persist) แม้ว่าจะมีการ refresh web browser ก็ตาม
  - สามารถดึงข้อมูลใน Redux store มาใช้ได้เหมือนเดิม เมื่อเปิด application ขึ้นมาอีกครั้ง
  - Redux Persist มาพร้อม method ที่รองรับการ customize ข้อมูล state ที่ถูก persist ไว้ และถูก rehydrate ไว้ โดยเรียกใช้งานได้ผ่าน API

- Persist Storage Engine นำเสนอรูปแบบการจัดเก็บเพื่อ persist ข้อมูล state ไว้
  - Local Storage เป็น storage ที่นิยมใช้ร่วมกับ Redux Persist โดย import จาก library `redux-persist/lib/storage`
  - Session Storage เป็น storage ที่บริหารจัดการใน web browser เช่นเดียวกับ local storage แต่มีอายุการจัดเก็บ โดยถูก clear เมื่อสิ้นสุด page session import จาก `redux-persist/lib/storage/session`
  - Cookie Storage ผ่าน Redux Persist Cookie Storage Adapter ทำงานใน browser และ Node.js
    - https://github.com/abersager/redux-persist-cookie-storage

Redux Persist



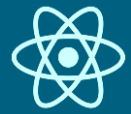
ในการใช้งาน Redux Persist นั้น developer ต้องปรับรูปแบบการทำงานของ Redux เพื่อใช้งานองค์ประกอบต่อไปนี้

- Persist Reducer เป็นการปรับแต่ง Reducer พื้นฐาน ตามรูปแบบ configuration ที่ระบุ เพื่อให้ persist Reducer นั้น ๆ ลงใน storage

```
persistedReducer = persistReducer(persistConfig, reducer)
```

  - *persistConfig* Configuration สำหรับ Redux Persist เป็น configuration object ที่ประกอบด้วย attribute ต่าง ๆ ตัวอย่างเช่น
    - *key* ชื่อ key ที่ใช้สำหรับการ persist
    - *storage* Object ของ storage adapter
    - *version* เลขเวอร์ชั่น
  - *reducer* ที่ต้องการให้รองรับการ persist
  - *persistedReducer* Persist Reducer ผลลัพธ์

Redux Persist



- **Persist Reducer** อาจสร้างจาก Reducer ที่รวมมาจากหลาย Reducer ได้ เช่น อาจรวมหลาย Reducer เข้าด้วยกันผ่านฟังก์ชั่น `combineReducers` ซึ่งเป็น helper function ซึ่งแปลง reducing function หลายฟังก์ชั่น เข้าเป็น single reducing function เดียว

```
rootReducer = combineReducers({k1: reducer1, k2: reducer2, …})
```

- *reducer1, reducer2* Child Reducer Function ที่จะนำมารวม
- *k1, k2* key ที่ใช้ระบุ Reducer Function
- *rootReducer* Reducer ผลลัพธ์ จากการรวม
- การเรียก Reducer ผลลัพธ์ จะเป็นการเรียก Child Reducer Function ทุกตัว แล้วรวมผลเข้าไว้ใน state object

```
rootReducer = combineReducers({potato: potatoReducer, tomato: tomatoReducer})
```

รวม reducer ผ่าน `combineReducers`

```
{
  potato: {
    // ... potatoes, and other state managed by the potatoReducer
  },
  tomato: {
    // ... tomatoes, and other state managed by the tomatoReducer
  }
}
```

โครงสร้างของ `state object` ผลลัพธ์

Redux Persist



- **Persist Store** เป็นการปรับแต่ง Store ให้รองรับการ persist สร้างโดย

```
persistor = persistStore(store,[config, callback])
```

  - *store* Redux Store ที่ต้องการจะ persist
  - *config* Optional configuration เช่น {manualPersist: true} เพื่อกำหนด ไม่ให้การ persist store เกิดทันทีหลังการเรียก persistStore เช่นกรณีที่ store อาจยังไม่พร้อมใช้งาน โดยให้เรียกแบบ manual ผ่านคำสั่ง *peristor*.persist()
  - *callback* Callback function ที่จะถูกเรียกหลังเสร็จสิ้นการทำ rehydration
  - *persistor* Persistor Object ผลลัพธ์

- **Persist Object** เป็น object ที่ได้มาจากการสร้าง persistStore โดยรองรับเมธอดต่อไปนี้
  - *peristor*.purge() Purge ข้อมูล state ออกจาก disk
  - *peristor*.flush() เขียนข้อมูล state ที่ pending อยู่ ลง disk
  - *peristor*.pause() หยุดการ persist
  - *peristor*.persist() เริ่มหรือ resume การ persist

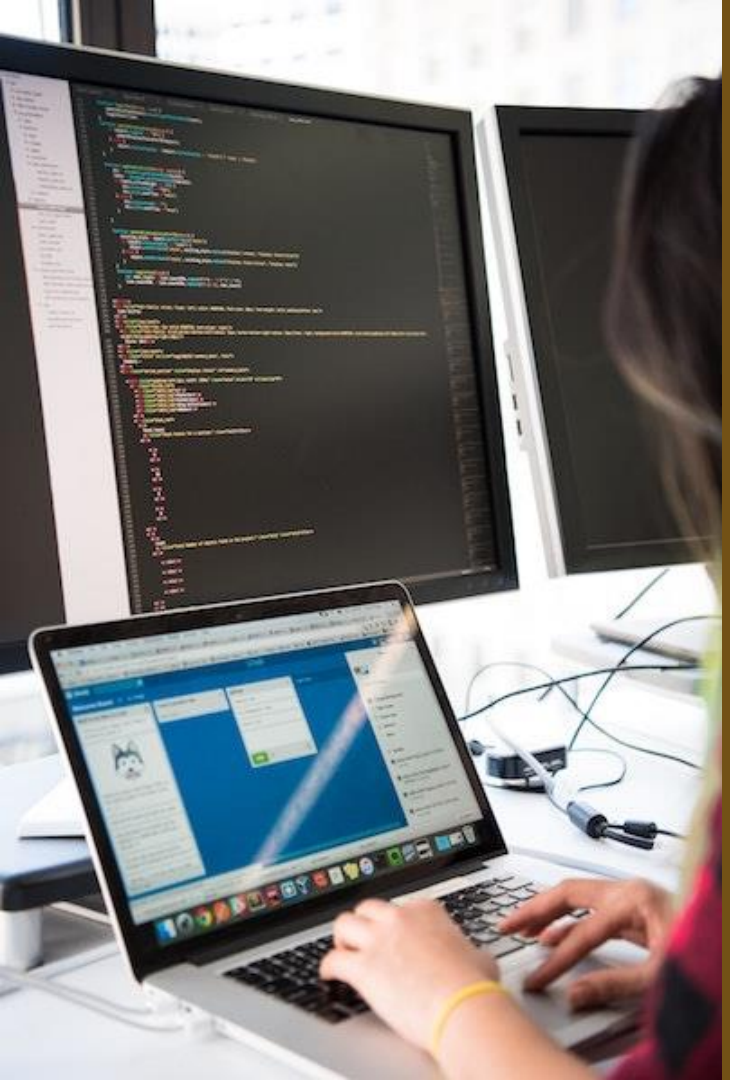
Redux Persist



- **Persist Gate** เป็นกลไกที่ทำงานกับ React Component เพื่อ delay การ Render UI จนกว่า state ที่ถูก persist ไว้ จะได้รับการดึงออกมา และ save เข้าไปใน redux
  - ใช้งานโดยการ wrap React Component ไว้ใน `<PersistGate>` … `</PersistGate>` โดยนิยม wrap root component เพื่อให้ครอบคลุมทุก UI component
  - `loading` prop ระบุ react component ที่จะแสดงระหว่างช่วง loading หรืออาจเป็น null
  - `persistor` prop ระบุ persistor object

ตัวอย่างการใช้งาน

```
<Provider store={store}>
  <PersistGate loading={null} persistor={persistor}>
    <RootComponent />
  </PersistGate>
</Provider>
```



Workshop 12.1: Redux Persist

Using Redux Persist



- ติดตั้ง Redux Persist version

```
npm i redux-persist
```

- สำหรับ Next.js app ซึ่งเน้น SSR เมื่อใช้ Redux Persist v.6.0 ซึ่งมี local storage เป็น default storage เราจะต้องสร้างเงื่อนไขในการเลือกใช้ storage
- ในไฟล์ src\redux\store.ts

```ts
import { configureStore, combineReducers } from "@reduxjs/toolkit";
import cartSlice from "./features/cartSlice";
import { useSelector, TypedUseSelectorHook } from "react-redux";
import { persistReducer } from "redux-persist";
import createWebStorage from 'redux-persist/lib/storage/createWebStorage';
import { WebStorage } from 'redux-persist/lib/types';

function createPersistStorage(): WebStorage {
    const isServer = typeof window === 'undefined';
    // Returns noop storage.
    if (isServer) {
      return {
        getItem() {
          return Promise.resolve(null);
        },
        setItem() {
          return Promise.resolve();
        },
        removeItem() {
          return Promise.resolve();
        },
      };
    }
    return createWebStorage('local');
}
const storage = createPersistStorage()
```

สร้าง function ที่ระบุเงื่อนไขการสร้าง storage ให้ไม่มีการทำงานอะไร (no op) หากเป็น server-side แต่มิเช่นนั้น ให้ใช้ local web storage

สร้าง storage

Using Redux Persist



- ในไฟล์ src\redux\store.ts

```
const storage = createPersistStorage()

const persistConfig = {
    key: "rootPersist",
    storage
}
const rootReducer = combineReducers({cartSlice})
const reduxPersistedReducer = persistReducer(persistConfig, rootReducer)

export const store = configureStore({
    reducer: reduxPersistedReducer
})

export type RootState = ReturnType<typeof store.getState>
export type AppDispatch = typeof store.dispatch
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector
```

ให้ reduxPersistedReducer เป็น reducer ของ store

Using Redux Persist



- เมื่อใช้ Redux Persist v.6.0+ การ dispatch action เช่น PERSIST มีการส่ง non-serializable value ให้ใช้ middleware เพื่อให้ ignore การ check non-serializable value ของ action
  - อ้างอิง : https://redux-toolkit.js.org/usage/usage-guide#use-with-redux-persist
- ในไฟล์ src\redux\store.ts

```
import { configureStore, combineReducers } from "@reduxjs/toolkit";
import cartSlice from "./features/cartSlice";
import { useSelector, TypedUseSelectorHook } from "react-redux";
import { persistReducer, FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER } from "redux-persist";
import createWebStorage from 'redux-persist/lib/storage/createWebStorage';
import { WebStorage } from 'redux-persist/lib/types';

function createPersistStorage(): WebStorage {
```

```
export const store = configureStore({
    reducer: reduxPersistedReducer,
    middleware: (getDefaultMiddleware) => getDefaultMiddleware({
        serializableCheck: {
          ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER],
        },
      })
})
```

Using Redux Persist



- ในไฟล์ src\redux\ReduxProvider.tsx

```tsx
"use client"
import { store } from '@/redux/store'
import {Provider as ReactReduxProvider} from 'react-redux'
import { persistStore } from 'redux-persist'
import { PersistGate } from 'redux-persist/integration/react'

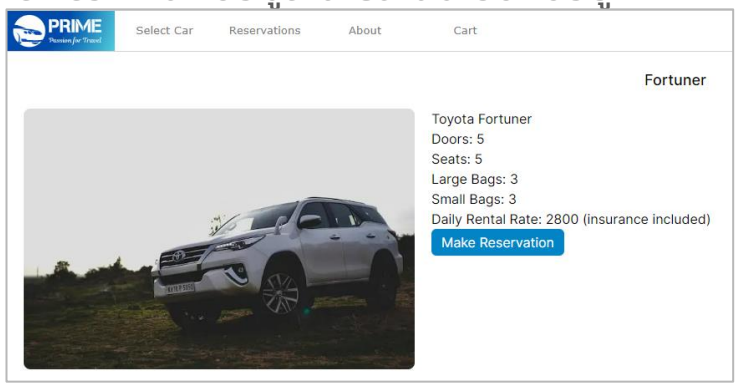
export default function ReduxProvider({children}: {children:React.ReactNode}) {

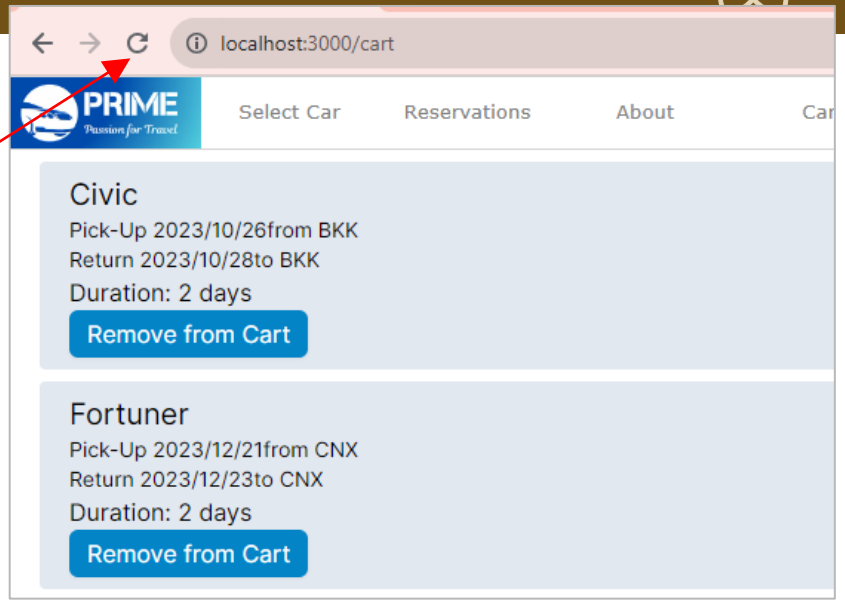
    let reduxPersistor = persistStore(store)

    return (
        <ReactReduxProvider store={store}>
        <PersistGate loading={null} persistor={reduxPersistor}>
            {children}
        </PersistGate>
        </ReactReduxProvider>
    )
}
```

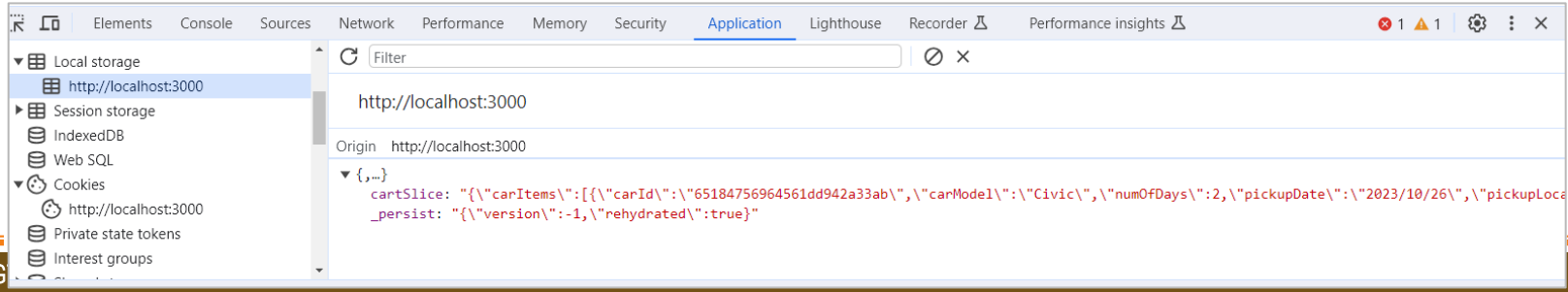
Using Redux Persist

- ทดสอบการใช้งาน http://localhost:3000
  - ลองจองรถลง Cart
  - แม้ว่าจะ refresh หน้า ข้อมูลใน Cart จะยังคงอยู่



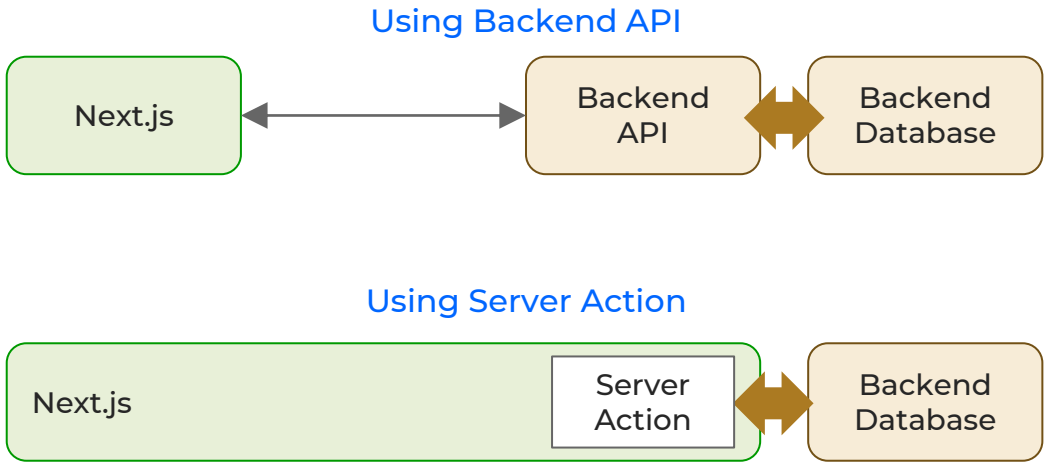


  - ดูใน Local Storage จะพบว่าข้อมูลถูก save ไว้ใน local storage



Server Action

- Next.js 13 มาพร้อมกลไก Server Actions
  - ช่วยให้การจัดการข้อมูลใน form เพื่อจัดส่งไปยัง backend server ทำได้ง่ายขึ้น โดยไม่จำเป็นต้องเรียกผ่าน Backend API
  - Developer สามารถนิยาม asynchronous server function ได้ภายใน component
  - รองรับการสร้าง `server-only` form ระบุการกระทำ Server Action ใน Server Component



Server Action



- Server Action ยังอยู่ในสถานะ experimental ในการใช้งาน Developer ต้อง enable Server Action ใน `next.config.js` โดย enable experimental `serverActions` flag

```
module.exports = {
  experimental: {
    serverActions: true,
  },
}
```

- สร้าง Server Action ได้ 2 รูปแบบ
  - สร้าง Server Action ไว้ใน Component โดยต้องเป็น Server Component เท่านั้น
  - สร้าง Server Action ไว้ในไฟล์แยก เพื่อให้ reuse ได้ โดยเรียกใช้ได้จากทั้ง Server Component และจาก Client Component
    - สามารถสร้างหลาย Server Action ไว้ในไฟล์เดียว
- Server Action สามารถกำหนดใช้ได้ ได้ทั้งกับ Server Component และ Client Component จึงมีความยืดหยุ่นต่อการใช้งาน
  - ใน Server Component นั้น form เรียก Server Action จาก Server Component โดยไม่อาศัย JavaScript
  - ใน Client Component นั้น form เรียก Server Action จะ queue submission

Server Action

- การใช้ Server Action บน Server Component
  - สร้าง Server Action โดยสร้าง asynchronous function ที่มี `"use server"` directive ที่ส่วนบนของ function body
  - `"use server"` ระบุให้ function นี้ execute บน server เท่านั้น

```
export default function ServerComponent() {
  async function myAction() {
    'use server'
    // ...
  }
}
```

  - ที่ `form` ระบุ `action` attribute ของ form เป็นฟังก์ชั่นที่จะเรียกเมื่อ submit form

```
<form action={myAction}>...</form>
```

Server Action

- การใช้ Server Action บน Client Component
  - สร้าง Server Action ในไฟล์แยก ที่มี "use server" directive ที่ส่วนบนของ file
  - import Server Action ไปใช้ใน Client Component
    - “use server” ด้านบนของ file ระบุให้ export ทั้งหมดใน file เป็น async server function ที่นำไปใช้ที่อื่นได้

app/actions.js

```
'use server'

export async function myAction() {
  // ...
}
```

app/client-component.jsx

```
'use client'

import { myAction } from './actions'

export default function ClientComponent() {
  return (
    <form action={myAction}>
      <button type="submit">Add to Cart</button>
    </form>
  )
}
```

Revalidating Cached Data



- **Revalidation** เป็นกระบวนการล้าง (purge) Data Cache และ re-fetching ข้อมูลล่าสุด เป็นกลไกที่เป็นประโยชน์เมื่อ data ถูกเปลี่ยน และต้องการดึงข้อมูลล่าสุด
  - **Time-based revalidation** ให้ revalidate data โดยอัตโนมัติ หลังช่วงเวลาที่กำหนด เป็นประโยชน์เมื่อ data ไม่ได้รับการ update บ่อย หรือความใหม่ของ data ไม่ใช่สิ่งสำคัญมากนักสำหรับ application
    - ระบุผ่าน `next.revalidate` option ของคำสั่ง `fetch` โดยระบุ cache lifetime เป็นวินาที

```
fetch('https://...', { next: { revalidate: 3600 } })
```

  - **On-demand revalidation** ให้ revalidate data เมื่อต้องการ เช่น เมื่อมีการ submit form เป็นประโยชน์เมื่อต้องการข้อมูลล่าสุดทันที
    - Tag-based On-demand revalidation ระบุ Tag ให้กับข้อมูลที่ fetch แล้วจึงเรียก revalidate จากชื่อ tag
    - Path-based On-demand revalidation เรียก revalidate จาก path ที่ต้องการ revalidate data ที่เกี่ยวข้องกับ path นั้น ๆ

Revalidating Cached Data



- **Tag-based On-demand Revalidation** อาศัยกลไก cache tagging system สำหรับ invalidate fetch request ที่ครอบคลุมหลาย route
  - ขณะเรียกคำสั่ง fetch ให้ระบุ `next.tags` option เพื่อ tag cache entries ด้วย tags ที่ระบุ

```
const res = await fetch('https://...', { next: { tags: ['collection'] } })
```

  - เมื่อต้องการทำ Cache Invalidation ให้เรียกใช้ `updateTag` เพื่อเพื่อทำให้ Cache Entries ทั้งหมดที่เกี่ยวข้องกับ Tag นั้นหมดอายุทันที (Expire) โดยการเรียกใช้ข้อมูลครั้งถัดไปจะดึงข้อมูลใหม่ก่อนส่งผลลัพธ์กลับ (Blocking Revalidation) หรือ `revalidateTag()` เพื่อทำ Revalidation แบบ Stale-While-Revalidate (SWR)

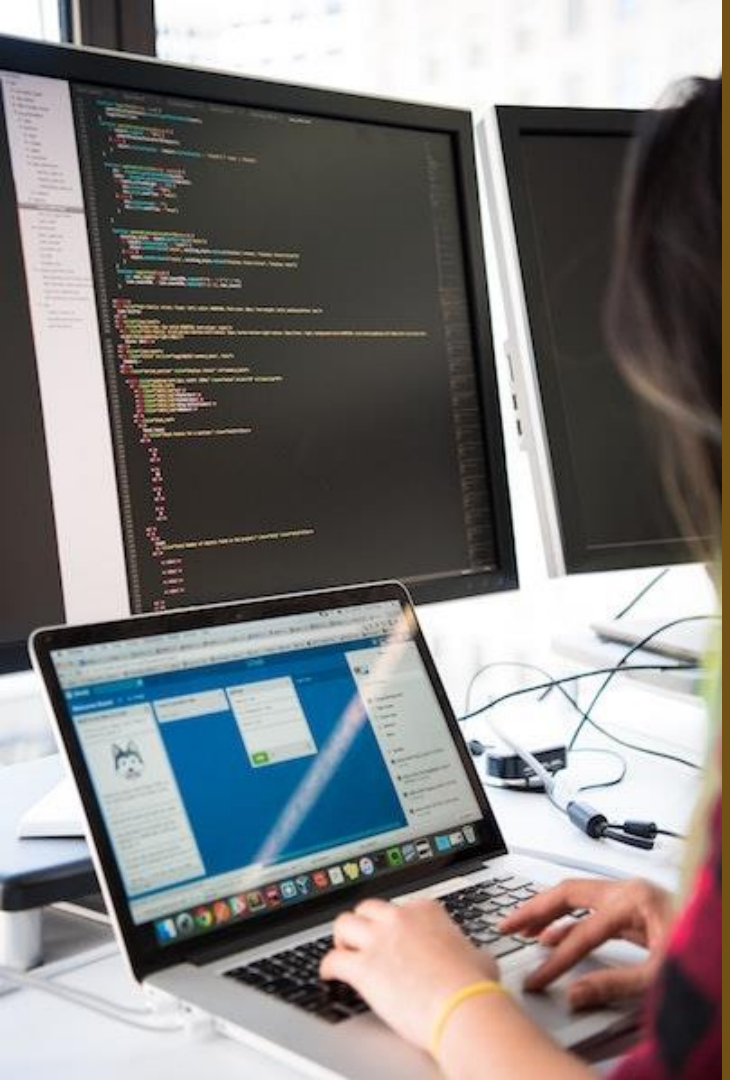
```
updateTag('collection')
```

- **Path-based On-demand Revalidation** เป็นการทำให้ Cache ที่เกี่ยวข้องกับ Path ที่ระบุหมดอายุ (Invalidate) ตามความต้องการ เพื่อให้ดึงข้อมูลและสร้างผลลัพธ์ใหม่เมื่อมีการเข้าถึง Path นั้นอีกครั้ง

```
import { revalidatePath } from 'next/cache'
revalidatePath('/blog/post-1')
```

```
import { revalidatePath } from 'next/cache'
revalidatePath('/blog/[slug]', 'page')
```

ระบุ `type` ได้ ว่าเป็น `page` หรือ `layout`



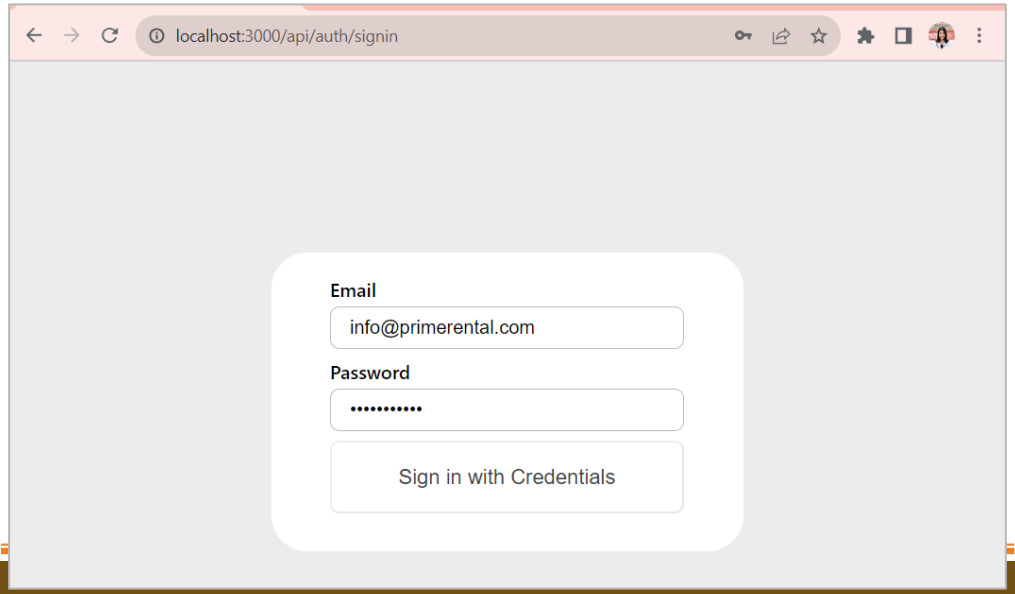
Workshop 12.2: Server Action

Adding Car Data with Server Action

- สร้าง account ของ admin
  - Email: info@primerental.com
  - Password: $0ftw@reD3v

- Sign-In ด้วย account ของ admin

```
{
  "name": "Info Staff",
  "email": "info@primerental.com",
  "tel": "0895521111",
  "role": "$0ftw@reD3v",
  "password": "admin"
}
```



Adding Car Data with Server Action



- ที่หน้า app\reservations\manage\@dashboard\page.tsx
  - แสดงส่วนให้เพิ่ม Car Data หาก user มี role เป็น admin

```
return (
    <main className="bg-slate-100 m-5 p-5">
        <div className="text-2xl">{profile.data.name}</div>
        <table className="table-auto border-separate border-spacing-2"><tbody>
            <tr><td>Email</td><td>{profile.data.email}</td></tr>
            <tr><td>Tel.</td><td>{profile.data.tel}</td></tr>
            <tr><td>Member Since</td><td>{createdAt.toString()}</td></tr>
        </tbody></table>
        {
            (profile.data.role=="admin")?
            <form>

            </form>
            :null
        }
    </main>
);
```

Adding Car Data with Server Action

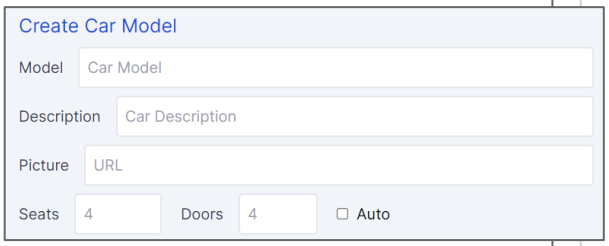


- ที่หน้า app\reservations\manage\@dashboard\page.tsx
  - สร้าง form สำหรับรับ Car Data

```
<form>
<div className="text-xl text-blue-700">Create Car Model</div>
<div className="flex items-center w-1/2 my-2">
    <label className="w-auto block text-gray-700 pr-4" htmlFor="model">
    Model</label>
    <input type="text" required id="model" name="model" placeholder="Car Model"
    className="bg-white border-2 border-gray-200 rounded w-full p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
</div>
<div className="flex items-center w-1/2 my-2">
    <label className="w-auto block text-gray-700 pr-4" htmlFor="desc">
    Description</label>
    <input type="text" required id="desc" name="desc" placeholder="Car Description"
    className="bg-white border-2 border-gray-200 rounded w-full p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
</div>
<div className="flex items-center w-1/2 my-2">
```

Adding Car Data with Server Action

- เพิ่ม form component




```
<div className="flex items-center w-1/2 my-2">
    <label className="w-auto block text-gray-700 pr-4" htmlFor="picture">
    Picture</label>
    <input type="text" required id="picture" name="picture" placeholder="URL"
    className="bg-white border-2 border-gray-200 rounded w-full p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
</div>
<div className="flex items-center w-1/2 my-2">
    <label className="w-auto block text-gray-700 pr-4" htmlFor="seats">
    Seats</label>
    <input type="number" required id="seats" name="seats" placeholder="4"
    min={0} max={50}
    className="bg-white border-2 border-gray-200 rounded w-auto p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
    <label className="w-auto block text-gray-700 pr-4 ml-5" htmlFor="doors">
    Doors</label>
    <input type="number" required id="doors" name="doors" placeholder="4"
    min={0} max={8}
    className="bg-white border-2 border-gray-200 rounded w-auto p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
    <input className="ml-5 mr-2" id="automatic" name="automatic" type="checkbox"/>
    <span>Auto</span>
</div>
```

Adding Car Data with Server Action



- เพิ่ม form component

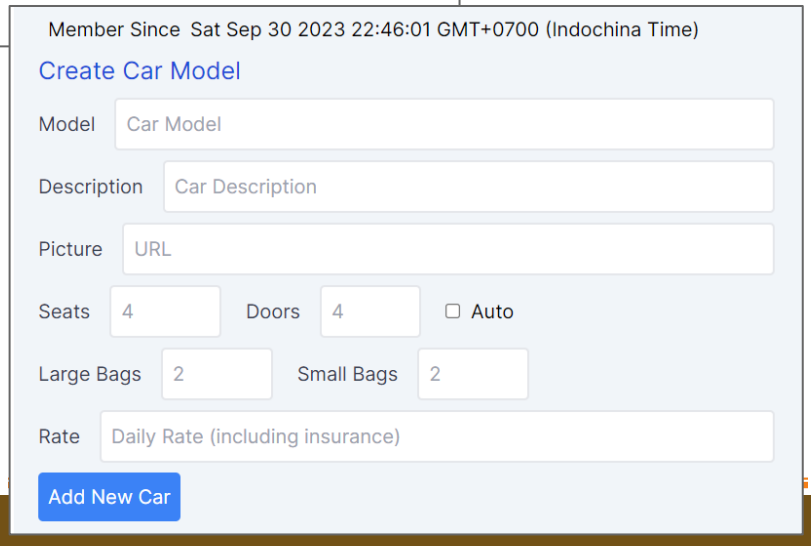
```
<div className="flex items-center w-1/2 my-2">
    <label className="w-auto block text-gray-700 pr-4" htmlFor="largebags">
    Large Bags</label>
    <input type="number" required id="largebags" name="largebags" placeholder="2"
    min={0} max={10}
    className="bg-white border-2 border-gray-200 rounded w-auto p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
    <label className="w-auto block text-gray-700 pr-4 ml-5" htmlFor="smallbags">
    Small Bags</label>
    <input type="number" required id="smallbags" name="smallbags" placeholder="2"
    min={0} max={10}
    className="bg-white border-2 border-gray-200 rounded w-auto p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
</div>
```

Adding Car Data with Server Action



```
<div className="flex items-center w-1/2 my-2">
    <label className="w-auto block text-gray-700 pr-4" htmlFor="dayRate">
    Rate</label>
    <input type="text" required id="dayRate" name="dayRate"
    placeholder="Daily Rate (including insurance)"
    className="bg-white border-2 border-gray-200 rounded w-full p-2
    text-gray-700 focus:outline-none focus:border-blue-400"/>
</div>
<button type="submit" className="bg-blue-500 hover:bg-blue-700
text-white p-2 rounded">Add New Car</button>
```

- เพิ่ม form component
- พิจารณา form ที่สร้างได้



Adding Car Data with Server Action



- ประกาศ Environment Variable เพื่อเก็บ Connection String จาก MongoDB Atlas แบบไฟล์ `config.env` ใน backend
  - ไฟล์ `.env.local`

```
PORT=5000
NODE_ENV=development
MONGO_URI=mongodb+srv://[redacted].mongodb.net/?appName=Cluster0
```

- ที่ console ติดตั้ง mongoose ใน Next.js project
  - `npm i mongoose`

Adding Car Data with Server Action



- สร้างโฟลเดอร์ src/db
- สร้างไฟล์ dbConnect.ts

```
import mongoose from "mongoose"

let isConnected = false
export const dbConnect =async () => {
    mongoose.set("strictQuery", true)
    if(isConnected) return

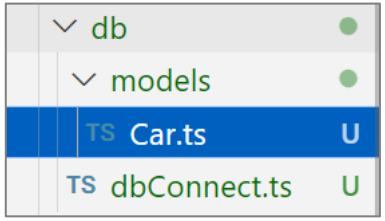
    const MONGO_URI = process.env.MONGO_URI
    if(!MONGO_URI) throw new Error("Please define MONGO_URI")

    try{
        await mongoose.connect(MONGO_URI, {bufferCommands: false})
        isConnected = true
        console.log("Connected")
    } catch(error) {
        console.log(error)
    }
}
```

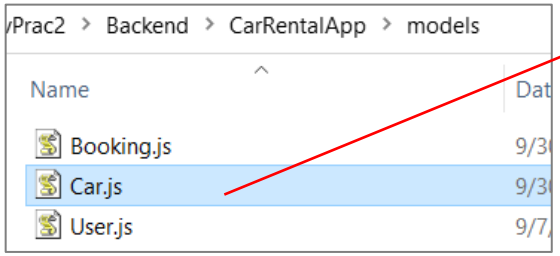
Adding Car Data with Server Action



- สร้างโฟลเดอร์ `src/db/models`
- สร้างไฟล์ `Car.ts` สำหรับเก็บ Schema ของ Car Model



- นำ Schema ของ Car มาจาก `models\Car.js` ใน Backend




```
1  import mongoose from "mongoose"
2  const CarSchema = new mongoose.Schema(
3    {
4      model: {···
10     },
11     description: {···
14     },
15     picture: {···
18     },
19     seats: {···
22     },
23     doors: {···
26     },
27     largebags: {···
30     },
31     smallbags: {···
34     },
35     automatic: {···
38     },
39     dayRate: {···
42     },
43   }
44 );
45 const Car = mongoose.models.Car || mongoose.model("Car", CarSchema);
46 export default Car
```

Export `Car` เพื่อนำไปใช้งานที่อื่น

Adding Car Data with Server Action



- ที่หน้า app/reservations/manage/@dashboard/page.tsx
  - สร้าง function addCar สำหรับเพิ่มข้อมูล car เข้าใน backend database เมื่อ Submit Form
  - ใช้ keyword “use server” เพื่อระบุว่าฟังก์ชั่นนี้ทำงานบน Server เท่านั้น

```
export default async function DashboardPage () {
    const addCar =async (addCarForm:FormData) => {
        "use server"

    }

    const session = await getServerSession(authOptions)
```

Adding Car Data with Server Action



- ที่หน้า app\reservations\manage\@dashboard\page.tsx
  - ให้ form action เป็น function addCar

```
(profile.data.role=="admin")?
<form action={addCar}>
<div className="text-xl text-blue-700">Create Car Model</div>
<div className="flex items-center w-1/2 my-2">
    <label className="w-auto block text-gray-700 pr-4" htmlFor="model">
    Model</label>
```

  - ที่ function addCar นำข้อมูลจากแต่ละ form element ใน FormData ที่รับเข้ามา มาเก็บไว้ใช้งาน

```
const addCar =async (addCarForm:FormData) => {
    "use server"
    const model = addCarForm.get("model")
    const description = addCarForm.get("desc")
    const picture = addCarForm.get("picture")
    const seats = addCarForm.get("seats")
    const doors = addCarForm.get("doors")
    const largebags = addCarForm.get("largebags")
    const smallbags = addCarForm.get("smallbags")
    const automatic = true
    const dayRate = addCarForm.get("dayRate")

}
```

Adding Car Data with Server Action



- ที่หน้า app\reservations\manage\@dashboard\page.tsx
  - ที่ function addCar

```
import { authOptions } from "@/app/api/auth/[...nextauth]/route";
import { getServerSession } from 'next-auth';
import getUserProfile from "@/libs/getUserProfile";
import Car from "@/db/models/Car"
import { dbConnect } from "@/db/dbConnect";

export default async function DashboardPage () {
```

```
    const dayRate = addCarForm.get("dayRate")

    try {
        await dbConnect()
        const car = await Car.create({
            "model": model,
            "description": description,
            "picture": picture,
            "seats": seats,
            "doors": doors,
            "largebags": largebags,
            "smallbags": smallbags,
            "automatic": automatic,
            "dayRate": dayRate
        })
    } catch (error) {
        console.log(error)
    }
}
```

สร้าง Car จาก Car Schema

เรียก dbConnect เพื่อเชื่อมต่อกับ Backend Database

Adding Car Data with Server Action



- ที่ src\libs\getCars.tsx
  - เพิ่ม tag 'cars' ไปที่การ fetch ข้อมูลใน function getCars

```
export default async function getCars() {

    // add timeout for loading delay testing
    await new Promise((resolve)=>setTimeout(resolve, 5000))

    const response = await fetch("http://localhost:5000/api/v1/cars", { next: {tags:['cars']} })
    if(!response.ok) {
```

- ที่หน้า reservations\manage\@dashboard\page.tsx
  - ที่ function addCar หลังการ add ให้ Revalidate Tag และ Redirect ไปที่ route /car

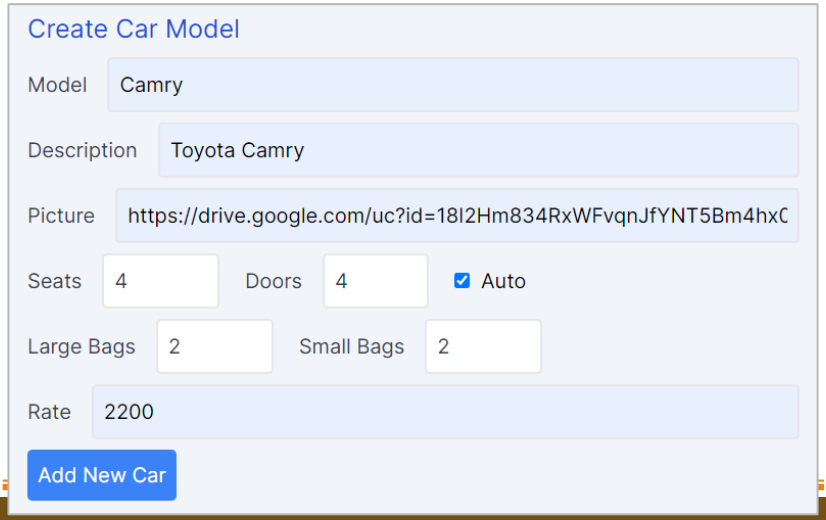
```
import { dbConnect } from '@/db/dbConnect'
import { updateTag } from 'next/cache'
import { redirect } from 'next/navigation'

export default async function DashboardPage () {
```

```
try {
    await dbConnect()
    const car = await Car.create({
        "model": model,
        "description": description,
        "picture": picture,
        "seats": seats,
        "doors": doors,
        "largebags": largebags,
        "smallbags": smallbags,
        "automatic": automatic,
        "dayRate": dayRate
    })
}
catch(error) {
    console.log(error)
}
updateTag("cars")
redirect("/car")
```

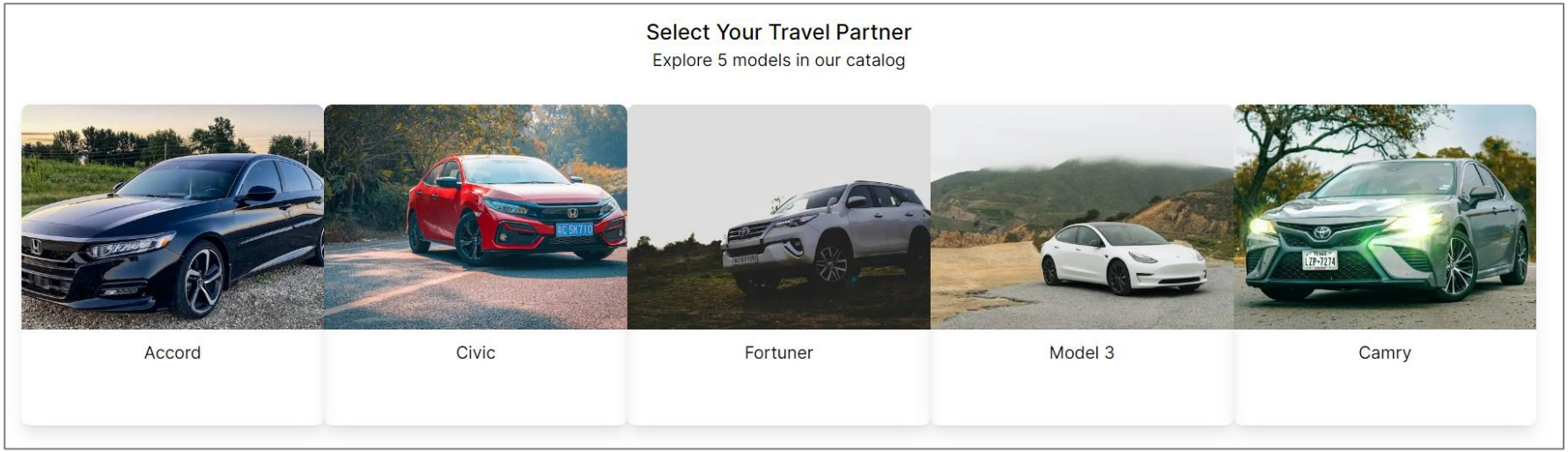
Adding Car Data with Server Action

- ทดสอบ Application โดยในขณะที่ Sign-In เป็น admin ให้ navigate ไปที่ http://localhost:3000/reservations/manage
  - สร้างข้อมูล Car Model ใหม่ ให้มีข้อมูลต่อไปนี้
    - Model: Camry
    - Description: Toyota Camry
    - Picture: https://drive.google.com/uc?id=18I2Hm834RxWFvqnJfYNT5Bm4hx0GqsAO
    - Seats: 4, Door: 4, Large Bags: 2, Small Bags: 2
    - Rate: 2200



Adding Car Data with Server Action

- เมื่อ redirect ไปที่หน้า http://localhost:3000/car
  - จะพบว่ามี Camry Car Model เพิ่มขึ้นมา



Tailwind Responsive Support

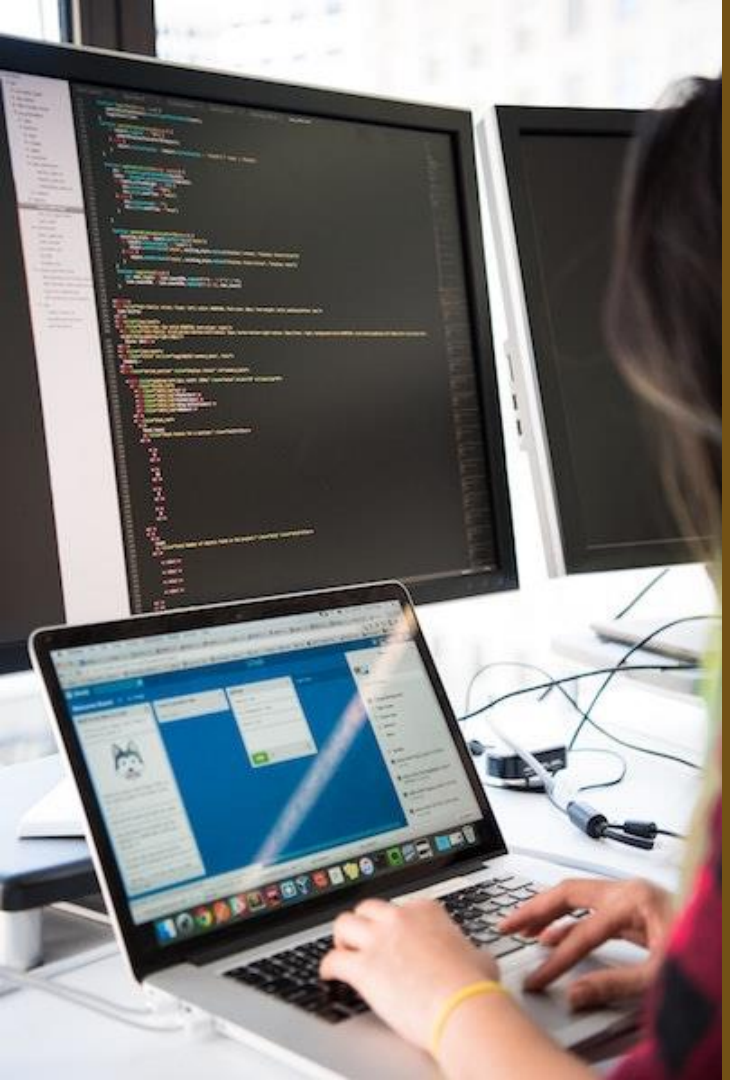


- ทุก utility class ใน Tailwind รองรับการนำไปใช้งานโดยมีเงื่อนไขตาม breakpoints ทำให้ง่ายต่อการสร้าง responsive interface
- โดย default ประกอบด้วย 5 breakpoint ซึ่งสอดคล้องกับ resolution ของอุปกรณ์ส่วนใหญ่
- ในการใช้งาน เจาะจง effect สำหรับแต่ละ breakpoint ผ่านการวางชื่อ breakpoint เป็น prefix สำหรับ utility นั้น ๆ ตามด้วยเครื่องหมาย :

```
<img class="w-16 md:w-32 lg:w-48" src="...">
```

- มี `width` เป็น 16 โดย default
- มี `width` เป็น 32 บน medium screen
- มี `width` เป็น 48 บน large screen

Breakpoint prefix	Minimum width
`sm`	640px
`md`	768px
`lg`	1024px
`xl`	1280px
`2xl`	1536px



Workshop 12.3:
Tailwind Responsive Support

Tailwind Responsive Support



- ที่หน้า src\components\CarCatalog.tsx
  - กำหนดความกว้างให้พื้นที่ Link ที่ครอบ ProductCard ตาม Resolution
  - กำหนด padding ให้พื้นที่ Link ที่ครอบ ProductCard ตาม Resolution

```
Explore {carJsonReady.count} models in our catalog
<div style={{margin:"20px", display:"flex",
    flexDirection:"row", alignContent:"space-around",
    justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
        {
            carJsonReady.data.map((carItem:Object)=>(
                <Link href={`/car/${carItem.id}`}
                  className="w-[100%] sm:w-[50%] md:w-[30%] lg:w-[25%]
                  p-2 sm:p-4 md:p-4 lg:p-8">
                  <ProductCard carName={carItem.model} imgSrc={carItem.picture}/>
                </Link>
            ))
        }
</div>
```

Tailwind Responsive Support

- Navigate ไปที่หน้า http://localhost:3000/car
  - จะพบว่าเป็น Responsive

